Minesweeper Robot: Software Architecture Documentation

June 25, 2026

Abstract

This document outlines the software architecture for an autonomous/teleoperated Minesweeper Robot designed for a 20x20 meter competition field. The system employs a dual-compute architecture utilizing a Raspberry Pi 4 for high-level ROS 2 logic and an ESP32 for real-time hardware interfacing. It features custom Kalman filtering for localization, sensor fusion for mine detection (combining analog metal detection with ONNX-based object detection), and a robust, Dockerized deployment environment. On the laptop, a dedicated ROS2-to-UDS bridge subscribes to all ROS 2 topics (including detection results from the ONNX node) and forwards consolidated telemetry to the Raylib GUI over a single Unix Domain Socket, fully decoupling the GUI from ROS.

Contents

1 System Overview **2**

2 Hardware Layer & Peripherals **2**

3 Architecture Diagram **2**

4 Communication & Protocols **2**
- 4.1 Pi ↔ ESP32 (UART) . . . 2
- 4.2 Robot ↔ Laptop (Wi-Fi) . . . 2
- 4.3 Laptop Internal Communication (Unix Domain Socket) . . . 3

5 Autonomy & Sensor Fusion **4**
- 5.1 Localization (Kalman Filter) . . . 4
- 5.2 Mine Detection (Decision Logic) . . . 4

6 Deployment Strategy **4**
- 6.1 Configuration Management . . . 4
- 6.2 Hardware Port Discovery Requirements . . . 5
- 6.3 Docker Deployment Requirements . . . 5

1 System Overview

The software stack is highly distributed, offloading heavy computational tasks (such as AI inference and GUI rendering) to a Base Station laptop, while reserving the onboard Raspberry Pi 4 for sensor data processing, filtering, and network streaming. The system is designed to be highly resistant to network latency and jitter, which are common in competition environments.

2 Hardware Layer & Peripherals

- **Raspberry Pi 4 (Main Onboard Compute):** Runs Ubuntu and ROS 2. Interacts with an I2C IMU, a USB Camera, and the ESP32 via UART.
- **ESP32 (Real-Time Controller):** Handles high-frequency hardware loops. Reads rotary encoders, samples the analog metal detector (via oversampling for noise reduction), and drives the dual-track/4-wheel motors, the arm servo/motor, and the electromagnet.
- **Base Station (Laptop):** Runs the Raylib C++ control station (UDS client), the ONNX object detection node (ROS 2 node publishing detection results), and the ROS2-to-UDS bridge that subscribes to all relevant ROS topics and exposes them over a Unix Domain Socket.

3 Architecture Diagram

4 Communication & Protocols

4.1 Pi ↔ ESP32 (UART)

Due to the unreliability of Micro-ROS over serial constraints, the Pi and ESP32 communicate via a custom byte-stuffing protocol using Consistent Overhead Byte Stuffing (COBS).

- **Data Integrity:** Packets are strictly delimited and verified via checksums to drop corrupted data instantly.
- **Memory Alignment:** All data structures transmitted over UART employ explicit compiler packing directives to prevent memory padding mismatches between the 32-bit ESP32 and 64-bit Pi architectures.
- **Watchdog Timer:** The ESP32 enforces a strict 250 millisecond timeout. If a velocity command packet is delayed due to expected network jitter, the robot coasts gracefully. If no packet is received beyond this threshold, the ESP32 zeroes the motor PWM to prevent runaway conditions.

4.2 Robot ↔ Laptop (Wi-Fi)

To conserve Wi-Fi bandwidth, raw video is expressly excluded from ROS 2 DDS. Instead, a lightweight pipeline hosts an RTSP (Real-Time Streaming Protocol) server on the Raspberry Pi. The laptop acts as an RTSP client, allowing OpenCV to pull the network stream natively and feed it to the ONNX inference model. This structure permits multiple clients to view the video feed (e.g., for debugging) without duplicating the network load.

All telemetry, control, and detection data flow over standard ROS 2 DDS **only** between the Raspberry Pi and the ROS2-to-UDS bridge, and between the ONNX node and the bridge. Specifically: - The Raspberry Pi publishes: - `/pose` – filtered robot pose from the Kalman filter. - `/detection_raw` – combined mine detection status (from the decision node, based on metal detector

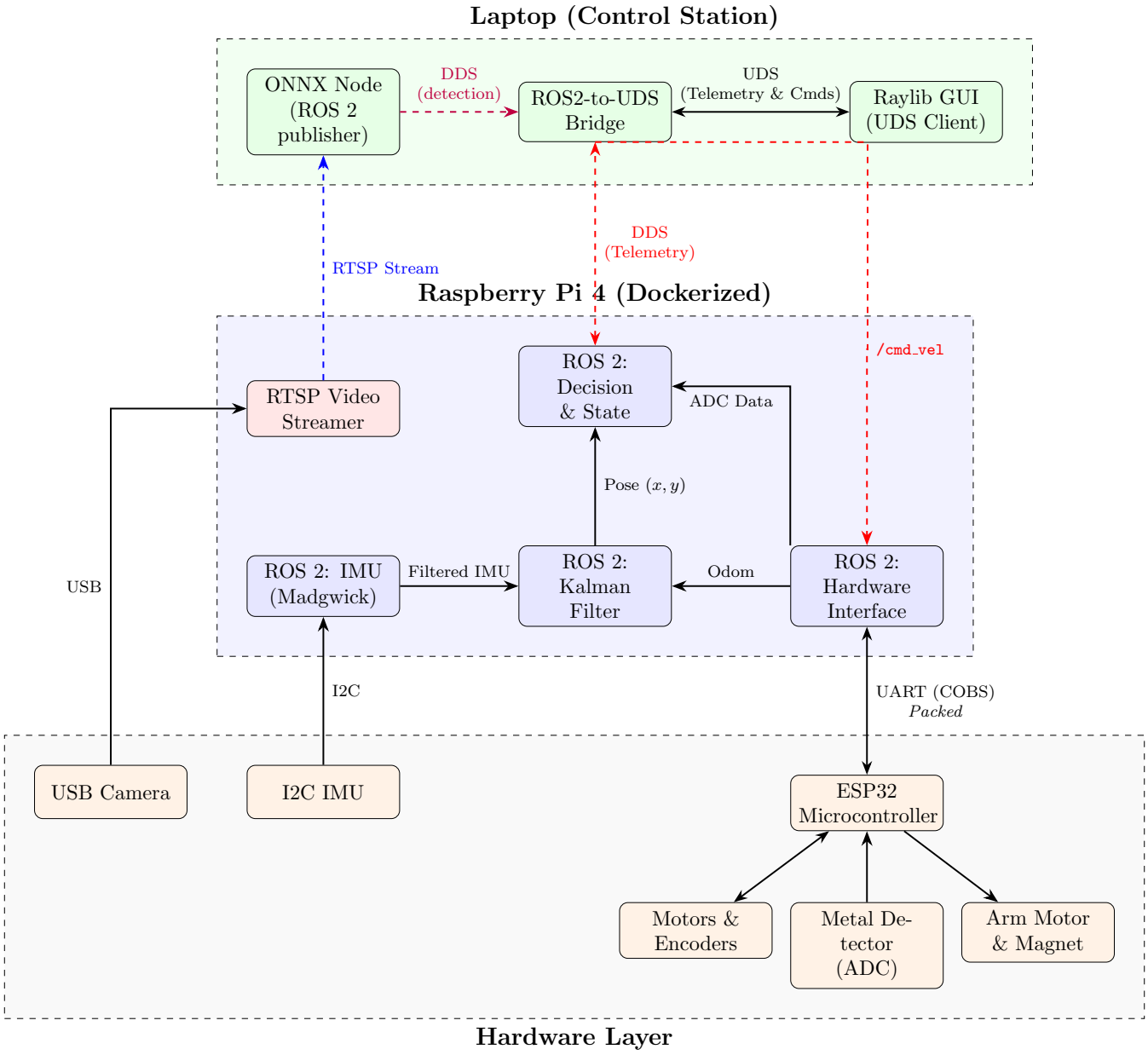


Figure 1: System Architecture Diagram

and visual confidence). - `/odom` – raw odometry (optional). - The ONNX node (running on the laptop) subscribes to the RTSP stream, runs inference, and publishes `/detection_vision` – a topic containing bounding boxes, class labels, and confidence scores. - The ROS2-to-UDS bridge subscribes to all these topics (from both the Pi and the ONNX node), consolidates the information, and forwards it to the Raylib GUI over a Unix Domain Socket. The bridge also receives velocity commands from the GUI via the same socket and publishes them as `/cmd_vel` to the Pi.

4.3 Laptop Internal Communication (Unix Domain Socket)

All communication between the ROS2-to-UDS bridge and the Raylib GUI occurs over a **single Unix Domain Socket (UDS)**, which provides:

- **Ultra-Low Latency:** UDS completely bypasses the network stack, providing high-bandwidth, near-zero latency IPC for high-frequency telemetry and control messages.
- **Decoupled Processing:** The bridge and the GUI run in separate processes. Heavy ROS 2 or ONNX workloads cannot freeze the graphical interface.

- **Consolidated Data Stream:** The bridge aggregates pose, detection flags, vision results, and other state information into a single, well-defined binary or JSON protocol, minimising the GUI's complexity.
- **Bidirectional Command Channel:** The same socket is used to send velocity commands and reset signals from the GUI back to the bridge, which then forwards them to the Pi over ROS 2.
- **Future Jetson Migration:** This architecture allows the entire bridge+ONNX stack to be moved to an onboard NVIDIA Jetson. In that scenario, the UDS would be replaced by a TCP proxy, but the GUI code remains unchanged because the protocol stays identical.

5 Autonomy & Sensor Fusion

5.1 Localization (Kalman Filter)

Absolute positioning (such as standard GPS) is excluded to reduce complexity and reliance on inherently inaccurate systems. Instead, localization relies on a custom Kalman filter that fuses:

1. **Wheel Odometry** collected by the ESP32 and forwarded to the hardware interface.
2. **Inertial Data** processed through a Madgwick filter from the onboard I2C IMU.

To counteract accumulated drift over the 20x20 meter field, the Base Station GUI provides a manual reset mechanism. Operators can use the physical grid corners as anchor points. When a reset command is issued (sent via the bridge over UDS), the Kalman node forcibly updates its position state and zeroes its covariance matrix to reflect absolute certainty.

5.2 Mine Detection (Decision Logic)

Mine detection is handled by a dedicated sensor fusion node on the Pi that evaluates two primary inputs: the raw analog signal of the metal detector (smoothed via a moving average on the ESP32) and the object detection confidence from the ONNX model.

- **Surface Mine Assessment:** If the analog signal exceeds the defined threshold AND the visual confidence is high, the system logs the coordinate, arrests movement, and sequences the magnetic arm to secure the object.
- **Underground Mine Assessment:** If the analog signal exceeds the threshold BUT the visual confidence is negligible, the coordinate is mapped as an underground threat and the arm sequence is bypassed.

The resulting detection flags are published on the `/detection_raw` topic, while the ONNX node publishes detailed vision data on `/detection_vision`. The bridge merges both streams for the GUI.

6 Deployment Strategy

Deployment on the Raspberry Pi is strictly containerized using Docker. The image is built directly from version control on the target hardware to avoid cross-compilation errors.

6.1 Configuration Management

All system parameters—such as Kalman tuning matrices, metal detector analog thresholds, and robot kinematics—are abstracted into a configuration file. This file resides on the Pi's host filesystem and is mapped as a bind volume into the Docker container. This separation guarantees that parameters can be adjusted directly at the competition site without necessitating a container rebuild.

6.2 Hardware Port Discovery Requirements

Because Linux dynamically assigns peripheral paths based on boot order, an automated discovery script must execute prior to launching the container. This script is required to:

1. Query the system to identify the ESP32 based on its specific USB vendor/product IDs.
2. Identify the active USB camera endpoint.
3. Export these dynamic hardware paths into a standard environment variables file, ensuring the container binds to the correct physical devices regardless of boot order.

6.3 Docker Deployment Requirements

The container orchestration must adhere to the following strict rules to ensure seamless operation:

- **Network Isolation Bypass:** The container must run in host networking mode. This is absolutely critical to allow the ROS 2 DDS multicast packets to discover and communicate with the laptop Base Station over Wi-Fi, and automatically exposes the RTSP server port to the local network.
- **Hardware Access:** The container must be granted adequate privileges or explicit device mappings to interact directly with the UART paths, the I2C bus, and the video hardware.
- **Runtime Parameter Injection:** The deployment process must pass the dynamically discovered hardware ports as launch arguments to the core ROS bring-up sequence.